

## AI Summer School - Day 2: Exploratory Data Analysis (EDA)

In this notebook, we explore household power consumption data using Python. We'll learn to inspect, clean, visualize, and understand patterns in the dataset.

### \* Step 1: Load the Dataset

We use `pandas` to read the household dataset and combine `Date` and `Time` columns into a `datetime` column.

```
```python
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv(
    'data/household_power_consumption.txt',
    sep=';', na_values='?',
    parse_dates={'datetime': ['Date', 'Time']],
    infer_datetime_format=True, low_memory=False
)

print('[ok] Data Loaded:', df.shape)
```
```

### \* Step 2: Inspect the Dataset

Check data types and statistical summaries.

```
```python
print(df.info())
print(df.describe(include='all'))
```
```

### \* Step 3: Handle Missing Values

Replace missing entries with the column median.

```
```python
print(df.isnull().sum())
df.fillna(df.median(numeric_only=True), inplace=True)
```
```

### \* Step 4: Convert Data Types

Convert large float64 columns to float32 for efficiency.

```
```python
num_cols = ['Global_active_power', 'Global_reactive_power', 'Voltage',
            'Global_intensity', 'Sub_metering_1', 'Sub_metering_2', 'Sub_metering_3']
```

```
df[num_cols] = df[num_cols].astype('float32')
```

```
'''
```

#### \* Step 5: Visualize Outliers

Use Seaborn boxplot to check for outliers.

```
```python
```

```
plt.figure(figsize=(12, 5))
```

```
sns.boxplot(data=df[num_cols])
```

```
plt.title('Boxplots of Numeric Features')
```

```
plt.show()
```

```
'''
```

#### \* Step 6: Explore Correlations

```
```python
```

```
sns.heatmap(df[num_cols].corr(), annot=True, cmap='coolwarm')
```

```
plt.title('Feature Correlation Matrix')
```

```
plt.show()
```

```
'''
```

#### \* Step 7: Plot Time-Series Trends

```
```python
```

```
df.set_index('datetime', inplace=True)
```

```
df['Global_active_power'].plot(figsize=(12, 4))
```

```
plt.title('Global Active Power Over Time')
```

```
plt.ylabel('kW')
```

```
plt.show()
```

```
'''
```

#### \* Step 8: Save Cleaned Dataset

```
```python
```

```
df.to_csv('data/household_power_cleaned.csv')
```

```
print('[saved] Saved!')
```

```
'''
```